

PGR 210 - Natural Language Processing

Lec10

By Huamin Ren

Huamin.ren@kristiania.no

Outline

- Creating memory in a neural net
- Building a recurrent neural net
- Data handling for RNNs
- Backpropagating through time (BPTT)

Outline

- *Creating memory in a neural net*
- Building a recurrent neural net
- Data handling for RNNs
- Backpropagating through time (BPTT)

Unrolled recurrent neural net

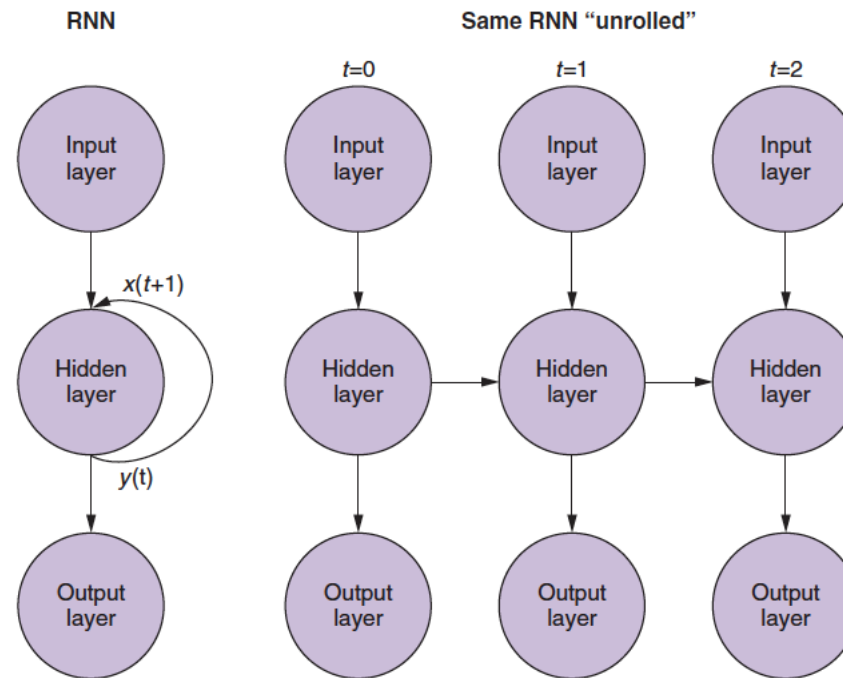


Figure 8.4 Unrolled recurrent neural net

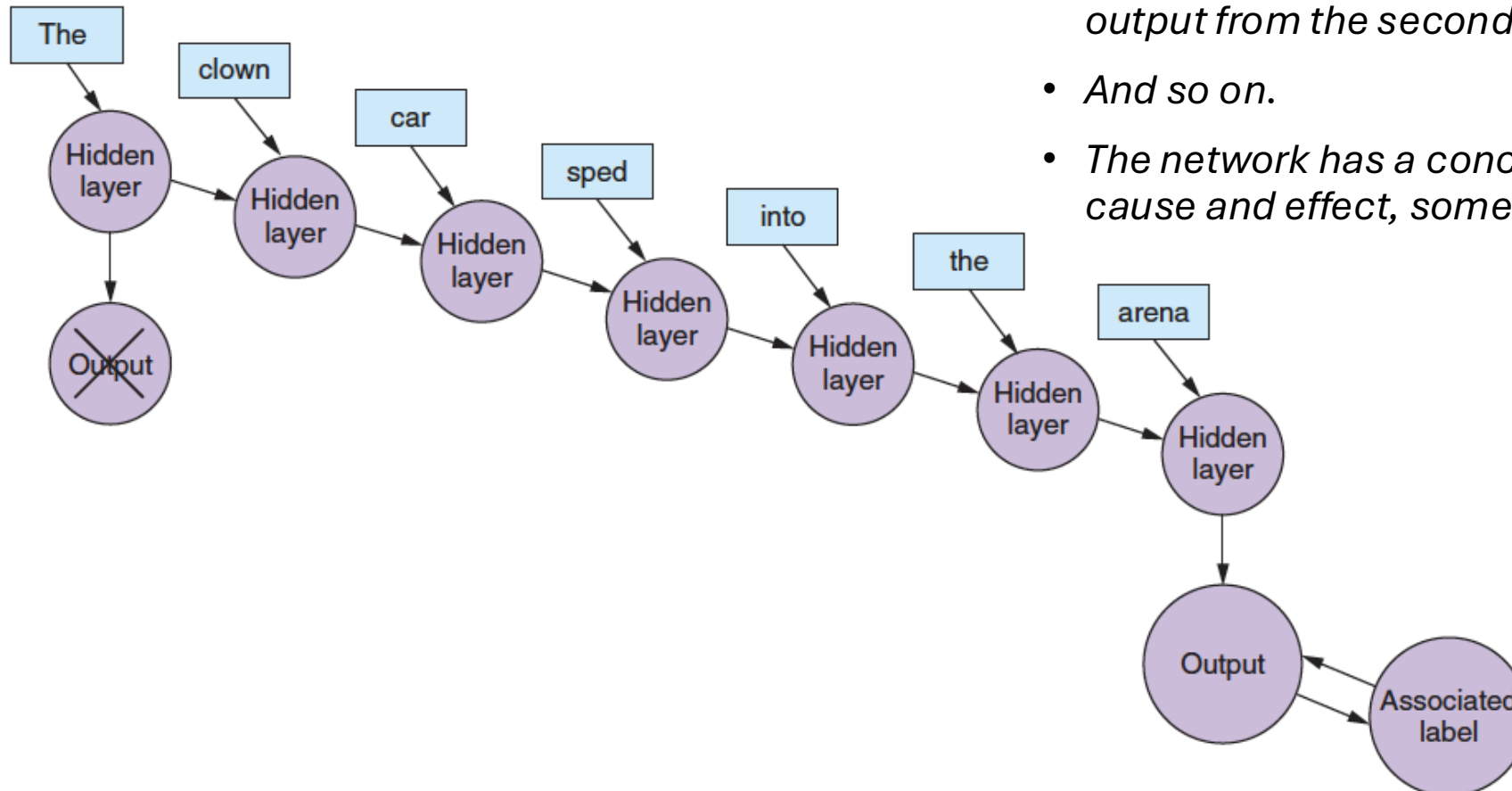
All the vertical paths in this visualization are clones, or views of the same neurons.

Recurrent Neural network

In your recurrent neural net, you pass in the word vector for the first token and get the network's output.

- *Then pass in the second token, but you also pass in the output from the first token!*
- *And then pass in the third token along with the output from the second token.*
- *And so on.*
- *The network has a concept of **before** and **after**, cause and effect, some vague notion of time...*

Recurrent neural net



Error? How about backpropagation?

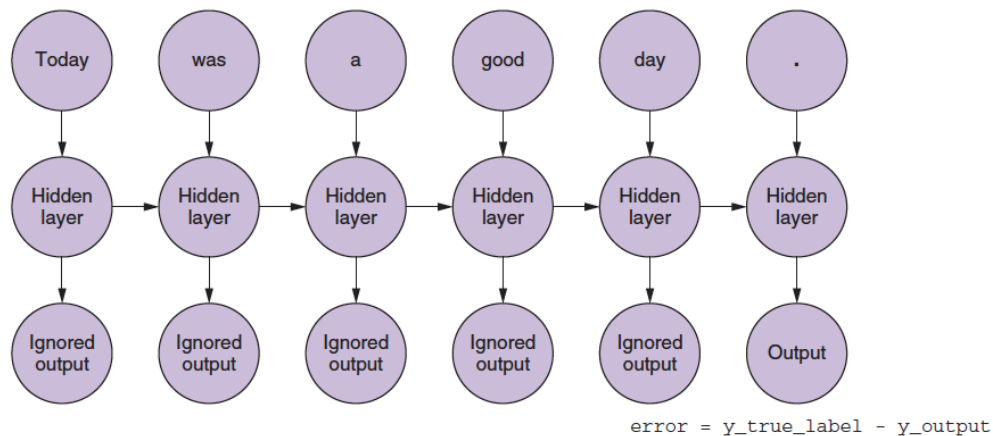


Figure 8.9 Only last output matters here

- For a given data sample, break it into smaller pieces that are fed into the network sequentially.
- Instead of dealing with the output generated by any of these “subsamples” directly, feed it back into the network.
- Only concerned with the final output, at least for now.
- Input each token from the sequence into your network and calculate the loss based on the output from the last time step (token) in the sequence.

Backpropagation

- Backpropagation: the correction to each weight is dependent on how much that weight contributed to the error
- What to do now (with time t)?

Read textbook

- 30 min

Outline

- Creating memory in a neural net
- Building a recurrent neural net
- Data handling for RNNs
- Backpropagating through time (BPTT)

Putting together

- Keras also comes with several tools, such as `model.summary()`, for inspection of the internals of your model.

```
num_neurons = 100

print('Build model...')
model = Sequential()

model.add(SimpleRNN(num_neurons, return_sequences=True, input_shape=(maxlen,
    embedding_dims)))
model.add(Dropout(.2))

model.add(Flatten())
model.add(Dense(1, activation='sigmoid'))

model.compile('rmsprop', 'binary_crossentropy', metrics=['accuracy'])
```

Layer (type)	Output Shape	Param #
simple_rnn_1 (SimpleRNN)	(None, 400, 100)	40,100
dropout_1 (Dropout)	(None, 400, 100)	0
flatten_1 (Flatten)	(None, 40000)	0
dense_1 (Dense)	(None, 1)	40,001

Total params: 80,101 (312.89 KB)

Trainable params: 80,101 (312.89 KB)

Non-trainable params: 0 (0.00 B)

Parameters

- Input dimension: 300 (?)
- Neurons: 100
- Weights for input x: 300×100
- Weights for hidden (w_h): 100×100
- Bias term before hidden: number of neurons, 100
- $30000 + 10000 + 100 = 40100$ for the first layer
- And so on....

Backpropagation through time (BPTT)

- Backpropagation Through Time For Networks With Long-Term Dependencies
- <https://arxiv.org/pdf/2103.15589v3>